

# การโปรแกรมเชิงพลวัต (2)

## Dynamic Programming

อ.ลือพล พิพานเมฆาภรณ์

# Content

- 0/1 Knapsack Problem
- All pair shortest path problem

# ทบทวน 0/1 knapsack problem

ถุงเป้รับน้ำหนักได้  $W_t$  กิโลกรัม มีของ  $n$  ชิ้น แต่ละชิ้นมีน้ำหนัก ( $w_i$ ) และมูลค่า ( $v_i$ ) ที่แตกต่างกัน จงหาวิธีเลือกของใส่ถุงเพื่อให้มีมูลค่ารวมมากที่สุด และน้ำหนักรวมไม่เกิน  $W_t$

| $w_i$ | $v_i$ |
|-------|-------|
| 1     | 1     |
| 3     | 4     |
| 4     | 5     |
| 5     | 7     |

wt = 7

$$\max \sum_{i \in T} v_i \quad \text{subject to} \quad \sum_{i \in T} w_i \leq W_t$$

# 0/1 knapsack problem

อัลกอริทึมเชิงละโมภ (greedy) เรียงลำดับของตาม density ( $v/w$ )

จะได้ 4 -> 2 -> 3 -> 1

| $w_i$ | $v_i$ |
|-------|-------|
| 1     | 1     |
| 3     | 4     |
| 4     | 5     |
| 5     | 7     |

total weight = 5 + 1 = 6 , total value = 7 + 1 = 8

แต่หากเลือก **item 2** และ **3** ใส่ลงไปในถุงจะได้มูลค่ารวมมากกว่า

wt = 7

Total weight = 3 + 4 = 7 , total value = 4+5 = 9

# Optimal sub-structure of 0/1 knapsack

ปัญหา 0/1 knapsack สามารถนิยามคำตอบของปัญหาในรูปของ recurrence relation ได้ โดยการแจกแจงคำตอบย่อยทั้งหมด (น้ำหนักที่ถุงเป่ารับได้ และมูลค่าของสินค้าแต่ละชิ้น)

$M(i, j)$  แทนมูลค่ารวมสูงสุดที่เลือกสินค้าตั้งแต่ชิ้นที่ 1 ถึง  $i$  ลงในถุงที่รับน้ำหนัก  $j$

โดย  $M(i, j)$  สามารถหาคำตอบได้ 2 กรณี

**กรณีที่ 1** ถ้าน้ำหนักของสินค้าชิ้นที่  $i$  ( $w_i$ ) มากกว่าน้ำหนักที่สามารถบรรจุในถุง  $j$  ดังนั้นสินค้า  $i$  จึงไม่เป็นส่วนหนึ่งของคำตอบ ส่งผลให้มูลค่ารวมมีค่าเท่ากับมูลค่ารวมของสินค้าก่อนหน้า  $i$  สำหรับน้ำหนักถุง  $j$

$$M(i, j) = M(i-1, j) \quad \text{ถ้า } w_i > j$$

ตัวอย่างเช่น สมมติใส่สินค้าไปแล้ว 2 ชิ้น เหลือน้ำหนักถุง 8 กิโล แต่สินค้าชิ้นที่ 3 น้ำหนัก 10 กิโล จึงไม่สามารถใส่ถุงได้ ดังนั้น

$$M(3, 8) = M(2, 8)$$

# Optimal sub-structure of 0/1 knapsack

กรณีที่ 2 ถ้าน้ำหนักของสินค้า  $w_i$  น้อยกว่าหรือเท่ากับน้ำหนักที่บรรจุในถุงได้  $j$  เราสามารถเลือกที่จะใส่สินค้า  $i$  หรือไม่ใส่ก็ได้

กรณี 2.1 หากเลือกใส่สินค้า  $i$  จะทำให้มูลค่ารวมเป็น

$$M(i, j) = V_i + M(i - 1, j - w_i)$$

ตัวอย่างเช่น สมมติใส่สินค้าไปแล้ว 2 ชิ้น เหลือน้ำหนักถุง 8 กิโล สินค้าชิ้นที่ 3 มูลค่า 10 น้ำหนัก 6 กิโล และเลือกใส่

$$M(3, 8) = 10 + M(2, 8-6) = 10 + M(2, 8)$$

กรณี 2.2 หากเลือกไม่ใส่  $i$  จะทำให้มูลค่ารวมเป็น

$$M(i, j) = M(i - 1, j)$$

ตัวอย่างเช่น กรณีเลือกไม่ใส่ชิ้นที่  $M(3, 8) = M(2, 8)$

เมื่อรวมกรณี 2.1 และ 2.2  $M(i, j) = \max \{ V_i + M(i-1, j-w_i), M(i-1, j) \}$

# Optimal sub-structure of 0/1 knapsack

สามารถเขียน recurrence relation ได้ ดังนี้

$$M(i, j) = \begin{cases} \max\{v_i + M(i - 1, j - w_i), M(i - 1, j)\} & w_i \leq j \\ M(i - 1, j) & w_i > j \end{cases}$$

ปัญหาย่อยที่เล็กที่สุด

$M(0, j) = 0$  หมายถึง มูลค่ารวมเป็น 0 เมื่อไม่มีของใส่ถุง

$M(i, 0) = 0$  หมายถึง มูลค่ารวมเป็น 0 เมื่อถุงรับน้ำหนักได้ 0

Wt = 7

V : 1 4 5 7

W: 1 3 4 5

$$M(i, j) = \begin{cases} \max\{v_i + M(i-1, j-w_i), M(i-1, j)\} & w_i \leq j \\ M(i-1, j) & w_i > j \end{cases}$$

เริ่มต้นด้วยการเตรียมตารางคำตอบและ เติมค่า 0 ลงในตารางตามเคสที่เล็กที่สุด

| i \ j  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|--------|---|---|---|---|---|---|---|---|
| (v, w) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (1, 1) | 0 |   |   |   |   |   |   |   |
| (4, 3) | 0 |   |   |   |   |   |   |   |
| (5, 4) | 0 |   |   |   |   |   |   |   |
| (7, 5) | 0 |   |   |   |   |   |   |   |



# Bottom-up Dynamic Programming

$$W_t = 7 \quad M(i, j) = \begin{cases} \max\{v_i + M(i-1, j-w_i), M(i-1, j)\} & w_i \leq j \\ M(i-1, j) & w_i > j \end{cases}$$

V : 1 4 5 7

W: 1 3 4 5

$$M(1, 1) = \max\{1 + M(0, 0), M(0, 1)\} = 1$$

$$M(1, 2) = \max\{1 + M(0, 1), M(0, 2)\} = 1$$

$$M(1, 3) = \max\{1 + M(0, 2), M(0, 3)\} = 1$$

| i \ j  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|--------|---|---|---|---|---|---|---|---|
| (v, w) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (1, 1) | 0 | 1 | 1 | 1 |   |   |   |   |
| (4, 3) | 0 |   |   |   |   |   |   |   |
| (5, 4) | 0 |   |   |   |   |   |   |   |
| (5, 7) | 0 |   |   |   |   |   |   |   |

# Bottom-up dynamic programming

$$W_t = 7 \quad M(i, j) = \begin{cases} \max\{v_i + M(i-1, j-w_i), M(i-1, j)\} & w_i \leq j \\ M(i-1, j) & w_i > j \end{cases}$$

V : 1 4 5 7

W: 1 3 4 5

| i \ j  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|--------|---|---|---|---|---|---|---|---|
| (v, w) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (1, 1) | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| (4, 3) | 0 |   |   |   |   |   |   |   |
| (5, 4) | 0 |   |   |   |   |   |   |   |
| (5, 7) | 0 |   |   |   |   |   |   |   |

$$W_t = 7 \quad M(i, j) = \begin{cases} \max\{v_i + M(i-1, j-w_i), M(i-1, j)\} & w_i \leq j \\ M(i-1, j) & w_i > j \end{cases}$$

V : 1 4 5 7

W: 1 3 4 5

$$M(2, 1) = M(1, 1) = 1 \quad M(2, 2) = M(1, 2) = 1$$

$$M(2, 3) = \max\{4 + M(1, 3-3), M(1, 3)\} = \max\{4, 1\} = 4$$

$$M(2, 4) = \max\{4 + M(1, 4-3), M(1, 4)\} = \max\{4+1, 1\} = 5$$

ทดลองคำนวณคำตอบ  $M(2, 5)$ ,  $M(2, 6)$  และ  $M(2, 7)$

| i \ j  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|--------|---|---|---|---|---|---|---|---|
| (v, w) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (1, 1) | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| (4, 3) | 0 | 1 | 1 | 4 | 5 |   |   |   |
| (5, 4) | 0 |   |   |   |   |   |   |   |
| (7, 5) | 0 |   |   |   |   |   |   |   |

Wt = 7

$$M(i, j) = \begin{cases} \max\{v_i + M(i-1, j-w_i), M(i-1, j)\} & w_i \leq j \\ M(i-1, j) & w_i > j \end{cases}$$

V : 1 4 5 7

W: 1 3 4 5

$$M(3, 1) = M(2, 1) = 1$$

$$M(3, 2) = M(2, 2) = 1$$

$$M(3, 3) = M(2, 3) = 4$$

$$M(3, 4) = \max\{5 + M(2, 4-4), M(2, 4)\} = \max\{5+0, 5\} = 5$$

| i \ j  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|--------|---|---|---|---|---|---|---|---|
| (v, w) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (1, 1) | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| (4, 3) | 0 | 1 | 1 | 4 | 5 | 5 | 5 | 5 |
| (5, 4) | 0 | 1 | 1 | 4 | 5 |   |   |   |
| (7, 5) | 0 |   |   |   |   |   |   |   |

$$w_t = 7 \quad M(i, j) = \begin{cases} \max\{v_i + M(i-1, j-w_i), M(i-1, j)\} & w_i \leq j \\ M(i-1, j) & w_i > j \end{cases}$$

V : 1 4 5 7

W: 1 3 4 5

$$\begin{aligned} M(4, 7) &= \max \{ 7 + M(3, 7-5), M(3, 7) \} \\ &= \max \{ 7+1, 9 \} = 9 \end{aligned}$$

| i \ j  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|--------|---|---|---|---|---|---|---|---|
| (v, w) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (1, 1) | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| (4, 3) | 0 | 1 | 1 | 4 | 5 | 5 | 5 | 5 |
| (5, 4) | 0 | 1 | 1 | 4 | 5 | 6 | 6 | 9 |
| (7, 5) | 0 | 1 | 1 | 4 | 5 | 7 | 8 | 9 |

# แบบฝึกหัด

1. จากปัญหา 0/1 knapsack ต่อไปนี้ จงเติมคำตอบลงในตาราง โดยวิธี **bottom-up**

Let  $W = 10$  and

| $i$   | 1  | 2  | 3  | 4  |
|-------|----|----|----|----|
| $v_i$ | 10 | 40 | 30 | 50 |
| $w_i$ | 5  | 4  | 6  | 3  |

[illegible]

# เฉลย

Let  $W = 10$  and

| $i$   | 1  | 2  | 3  | 4  |
|-------|----|----|----|----|
| $v_i$ | 10 | 40 | 30 | 50 |
| $w_i$ | 5  | 4  | 6  | 3  |

| $V[i, w]$ | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|-----------|---|---|---|----|----|----|----|----|----|----|----|
| $i = 0$   | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1         | 0 | 0 | 0 | 0  | 0  | 10 | 10 | 10 | 10 | 10 | 10 |
| 2         | 0 | 0 | 0 | 0  | 40 | 40 | 40 | 40 | 40 | 50 | 50 |
| 3         | 0 | 0 | 0 | 0  | 40 | 40 | 40 | 40 | 40 | 50 | 70 |
| 4         | 0 | 0 | 0 | 50 | 50 | 50 | 50 | 90 | 90 | 90 | 90 |

# แบบฝึกหัด

2. จงเขียนโปรแกรม **bottom-up DP** สำหรับปัญหาข้อที่ 1 จากนั้นแสดงตาราง **memory**

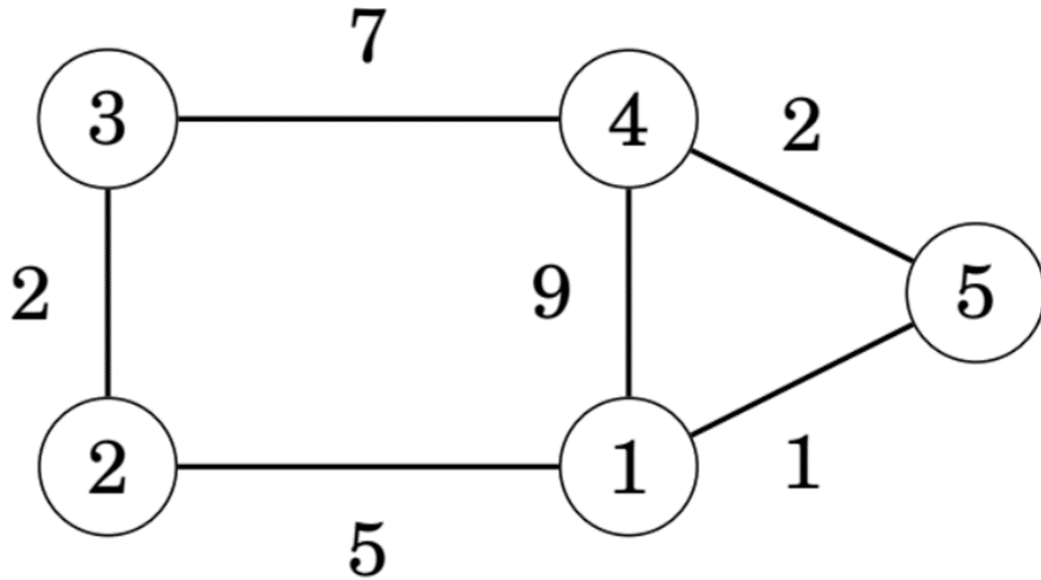
```
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 10 10 10 10 10 10
0 0 0 0 40 40 40 40 40 50 50
0 0 0 0 40 40 40 40 40 50 70
0 0 0 50 50 50 50 90 90 90 90
90
```

3. ปรับปรุงโค้ดข้อ 2 ให้เป็น **top-down DP** จากนั้นแสดงตาราง **memory**

```
0 0 0 0 0 0 0 0 0 0 0 0
0 0 10 40 10 0 0 0 0 0 0 10
0 40 0 0 0 0 0 0 0 0 40 50
0 0 0 0 0 0 0 0 0 0 70
0 0 0 0 0 0 0 0 0 0 90
90
```



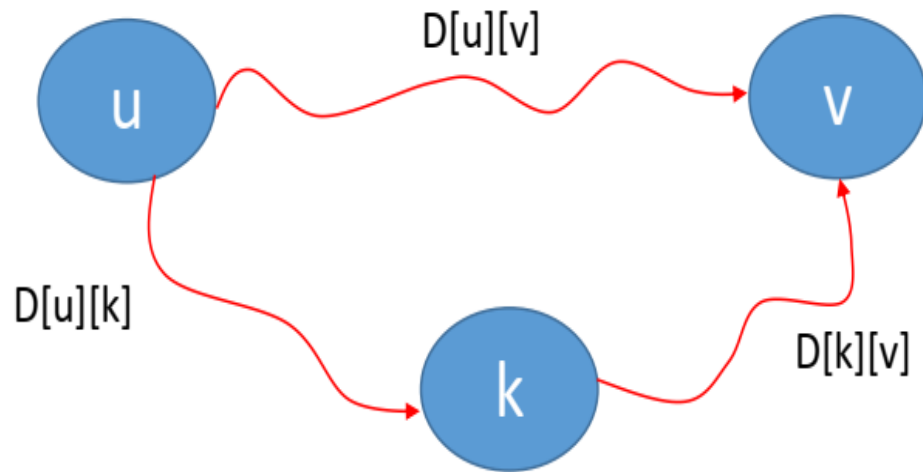
# ปัญหา All-pair shortest path



- เป็นปัญหาการหาเส้นทางที่สั้นที่สุดในกราฟถ่วงน้ำหนัก
- แตกต่างจากอัลกอริทึม **Dijkstra** ตรงที่ไม่ต้องการกำหนดจุดเริ่มต้น (**source vertex**) และผลลัพธ์จะสามารถเริ่มจากจุดใดก็ได้ในกราฟ

# อัลกอริทึม Floyd-Warshall

แนวคิดอัลกอริทึม Floyd คือจะสร้างเมตริกซ์ระยะทาง  $D$  (distance matrix) เพื่อใช้เก็บระยะทางที่สั้นที่สุดจาก  $u$  ไป  $v$  โดยที่  $u$  และ  $v$  เป็นสองเวอร์เท็กซ์ใดๆ ในกราฟ จากนั้นจะมีการอัปเดตระยะทางที่น้อยที่สุดในทุกๆ คู่ หากพบว่ามีความยาวที่สั้นกว่าระยะทางเดิมที่เคยบันทึกไว้



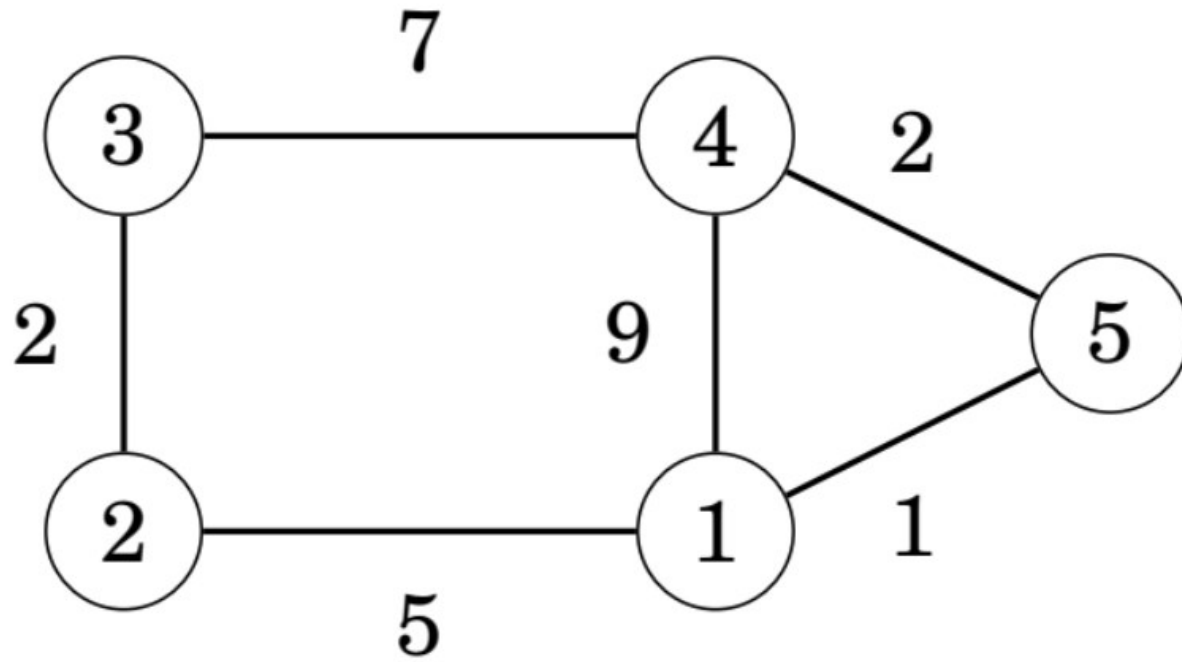
$$D[u][v] = \min \{ D[u][v], D[u][k] + D[k][v] \}$$

โดยที่  $k$  เป็น intermediate vertex

$k$  = เริ่มต้นจากเวอร์เท็กซ์ 0 ไปยังเวอร์เท็กซ์  $n-1$

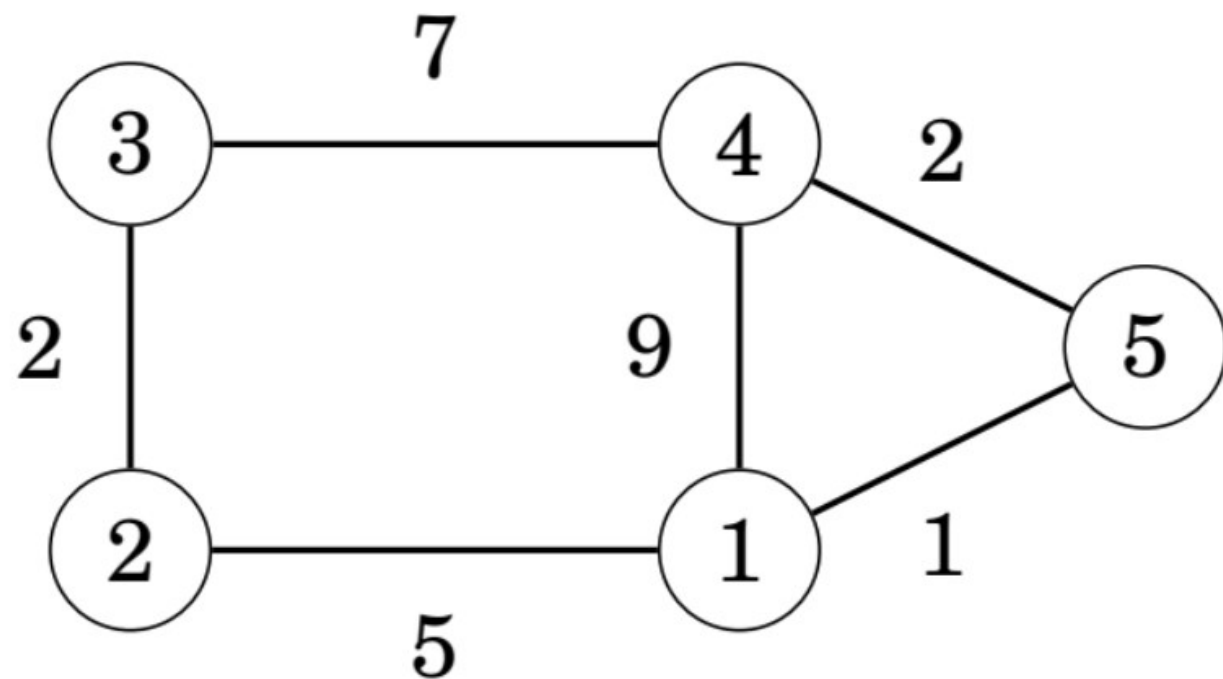
## ตัวอย่าง

- เตรียมเมตริกซ์ **D** โดยเริ่มต้นด้วยค่าระยะทางในแต่ละคู่ของเวอร์ทีกซ์ในกราฟ
- สำหรับ  $D[i][i] = 0$



|   | 1        | 2        | 3        | 4        | 5        |
|---|----------|----------|----------|----------|----------|
| 1 | 0        | 5        | $\infty$ | 9        | 1        |
| 2 | 5        | 0        | 2        | $\infty$ | $\infty$ |
| 3 | $\infty$ | 2        | 0        | 7        | $\infty$ |
| 4 | 9        | $\infty$ | 7        | 0        | 2        |
| 5 | 1        | $\infty$ | $\infty$ | 2        | 0        |

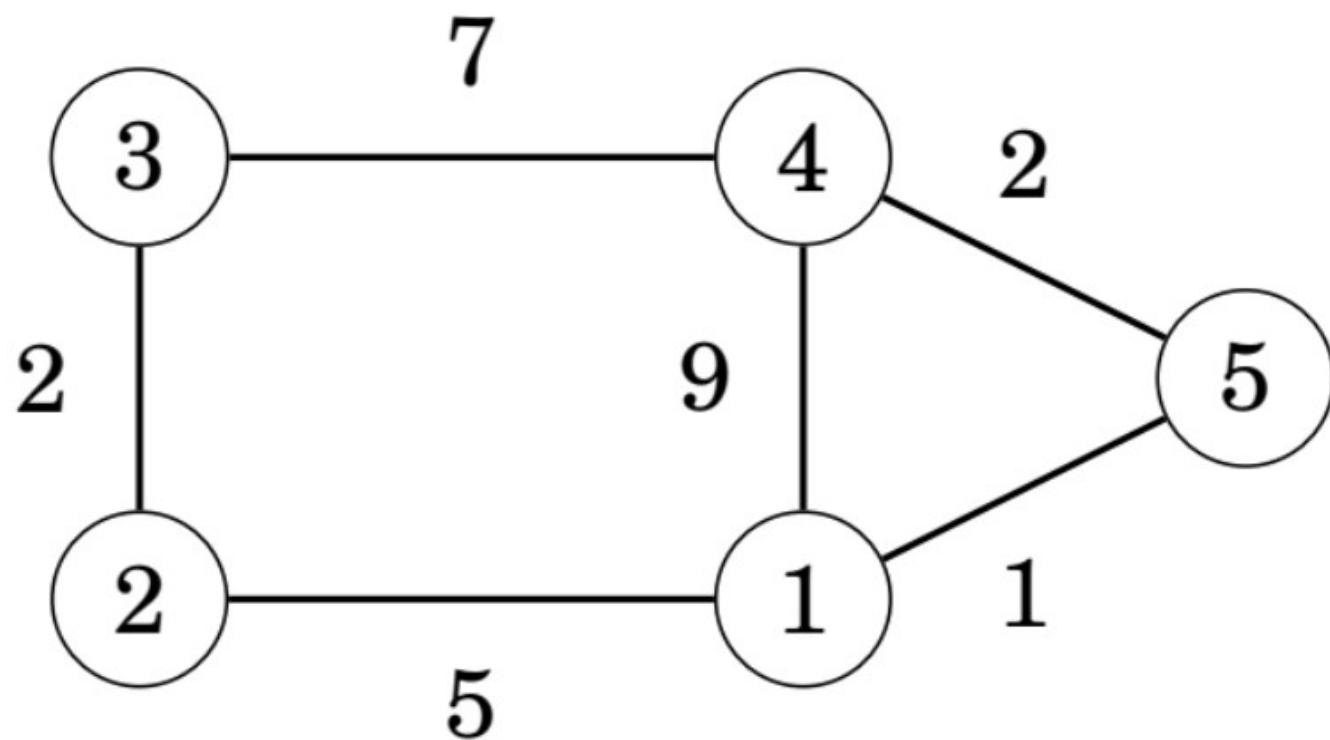
เวอร์เท็กซ์  $k = 1$



|   | 1        | 2        | 3        | 4        | 5        |
|---|----------|----------|----------|----------|----------|
| 1 | 0        | 5        | $\infty$ | 9        | 1        |
| 2 | 5        | 0        | 2        | $\infty$ | $\infty$ |
| 3 | $\infty$ | 2        | 0        | 7        | $\infty$ |
| 4 | 9        | $\infty$ | 7        | 0        | 2        |
| 5 | 1        | $\infty$ | $\infty$ | 2        | 0        |

|   | 1        | 2         | 3        | 4         | 5        |
|---|----------|-----------|----------|-----------|----------|
| 1 | 0        | 5         | $\infty$ | 9         | 1        |
| 2 | 5        | 0         | 2        | <b>14</b> | <b>6</b> |
| 3 | $\infty$ | 2         | 0        | 7         | $\infty$ |
| 4 | 9        | <b>14</b> | 7        | 0         | 2        |
| 5 | 1        | <b>6</b>  | $\infty$ | 2         | 0        |

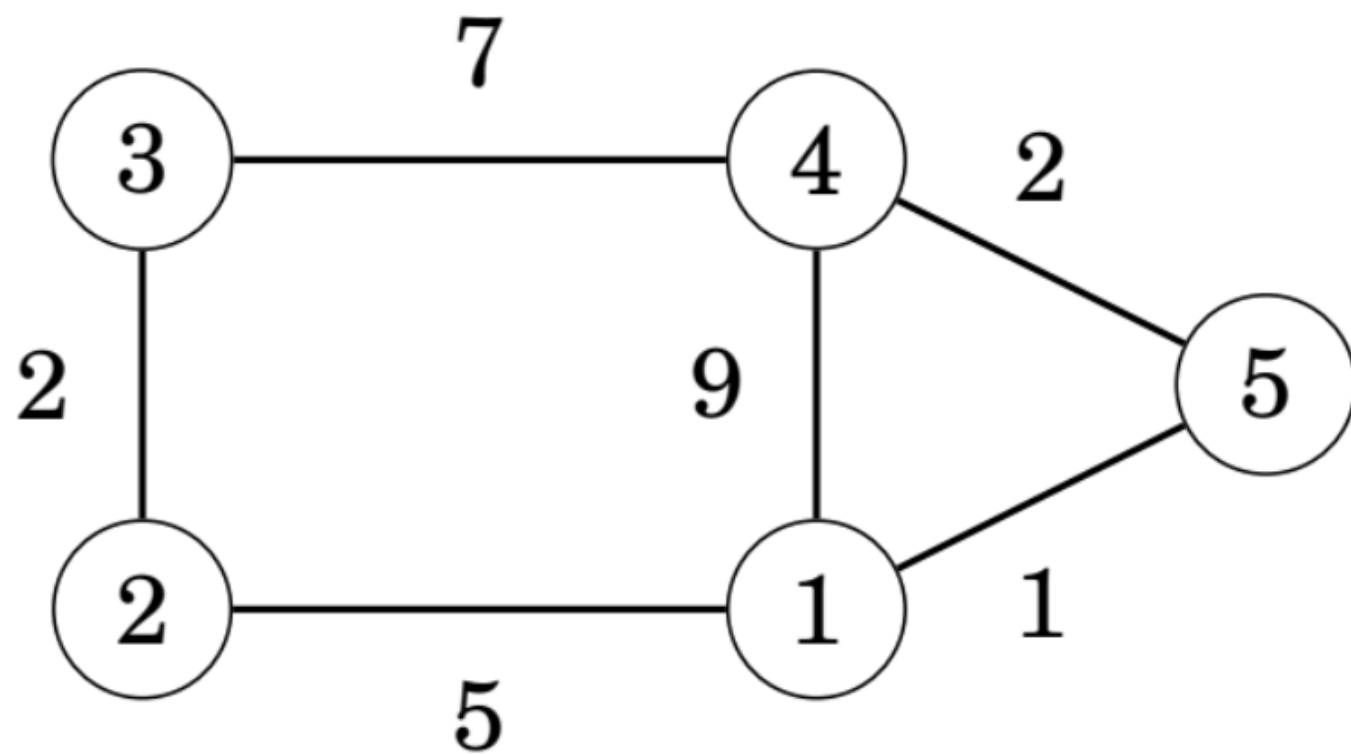
เวอร์เท็กซ์  $k = 2$



|   | 1        | 2  | 3        | 4  | 5        |
|---|----------|----|----------|----|----------|
| 1 | 0        | 5  | $\infty$ | 9  | 1        |
| 2 | 5        | 0  | 2        | 14 | 6        |
| 3 | $\infty$ | 2  | 0        | 7  | $\infty$ |
| 4 | 9        | 14 | 7        | 0  | 2        |
| 5 | 1        | 6  | $\infty$ | 2  | 0        |

|   | 1 | 2  | 3 | 4  | 5 |
|---|---|----|---|----|---|
| 1 | 0 | 5  | 7 | 9  | 1 |
| 2 | 5 | 0  | 2 | 14 | 6 |
| 3 | 7 | 2  | 0 | 7  | 8 |
| 4 | 9 | 14 | 7 | 0  | 2 |
| 5 | 1 | 6  | 8 | 2  | 0 |

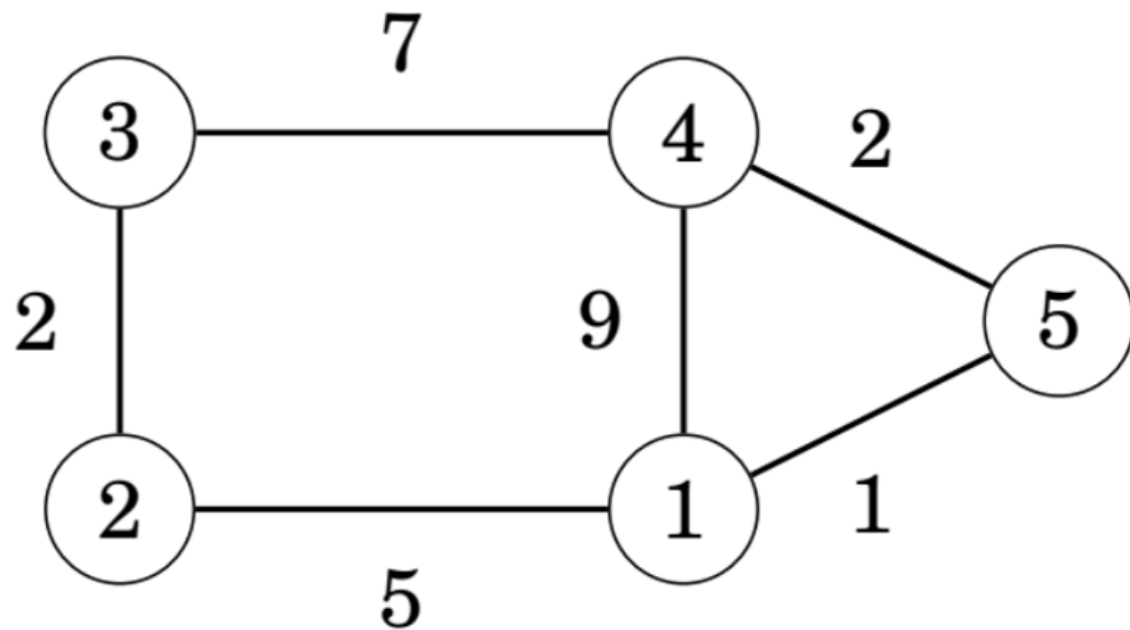
เวอร์เท็กซ์  $k = 3$



|   | 1        | 2  | 3        | 4  | 5        |
|---|----------|----|----------|----|----------|
| 1 | 0        | 5  | <b>7</b> | 9  | 1        |
| 2 | 5        | 0  | 2        | 14 | 6        |
| 3 | <b>7</b> | 2  | 0        | 7  | <b>8</b> |
| 4 | 9        | 14 | 7        | 0  | 2        |
| 5 | 1        | 6  | <b>8</b> | 2  | 0        |

|   | 1 | 2        | 3 | 4        | 5 |
|---|---|----------|---|----------|---|
| 1 | 0 | 5        | 7 | 9        | 1 |
| 2 | 5 | 0        | 2 | <b>9</b> | 6 |
| 3 | 7 | 2        | 0 | 7        | 8 |
| 4 | 9 | <b>9</b> | 7 | 0        | 2 |
| 5 | 1 | 6        | 8 | 2        | 0 |

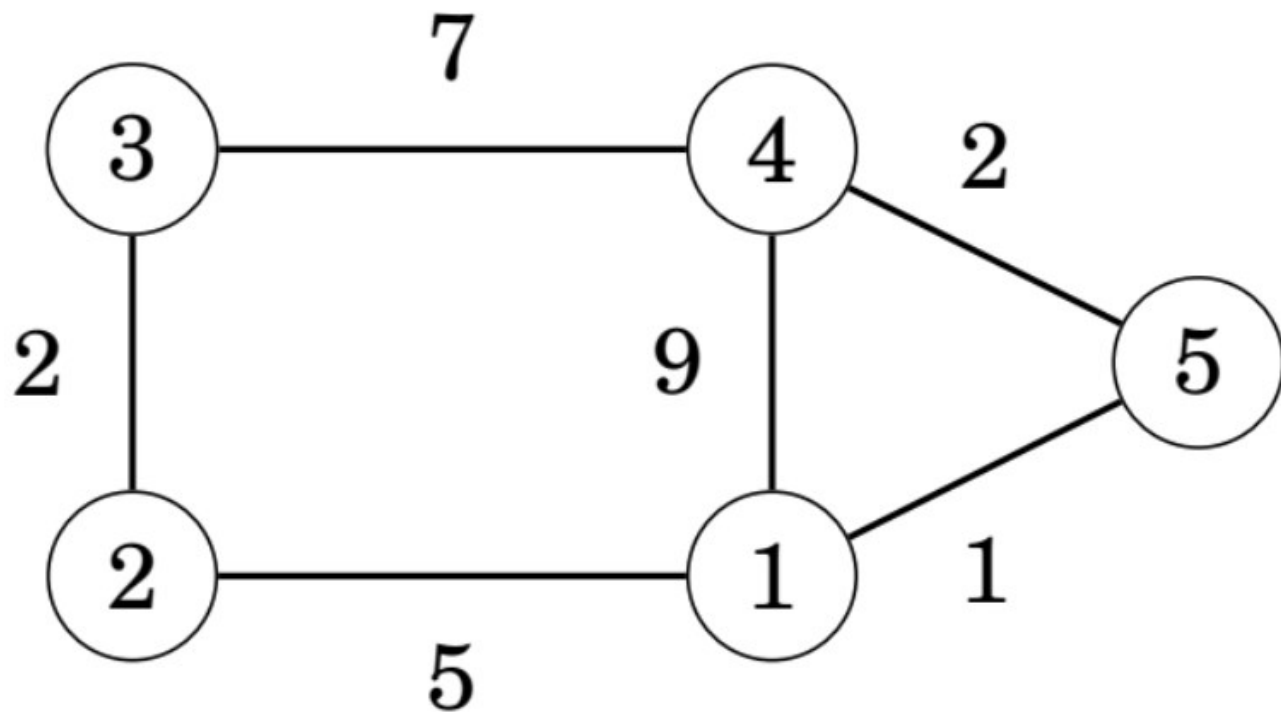
เวอร์เท็กซ์  $k = 4$



|   | 1        | 2        | 3 | 4        | 5 |
|---|----------|----------|---|----------|---|
| 1 | 0        | 5        | 7 | 9        | 1 |
| 2 | 5        | 0        | 2 | <b>9</b> | 6 |
| 3 | 7        | 2        | 0 | 7        | 8 |
| 4 | <b>9</b> | <b>9</b> | 7 | 0        | 2 |
| 5 | 1        | 6        | 8 | 2        | 0 |

|   | 1 | 2        | 3 | 4        | 5 |
|---|---|----------|---|----------|---|
| 1 | 0 | 5        | 7 | 9        | 1 |
| 2 | 5 | 0        | 2 | <b>9</b> | 6 |
| 3 | 7 | 2        | 0 | 7        | 8 |
| 4 | 9 | <b>9</b> | 7 | 0        | 2 |
| 5 | 1 | 6        | 8 | 2        | 0 |

# เวอร์เท็กซ์ $k = 5$



|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 5 | 7 | 9 | 1 |
| 2 | 5 | 0 | 2 | 9 | 6 |
| 3 | 7 | 2 | 0 | 7 | 8 |
| 4 | 9 | 9 | 7 | 0 | 2 |
| 5 | 1 | 6 | 8 | 2 | 0 |

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 5 | 7 | 3 | 1 |
| 2 | 5 | 0 | 2 | 8 | 6 |
| 3 | 7 | 2 | 0 | 7 | 8 |
| 4 | 3 | 8 | 7 | 0 | 2 |
| 5 | 1 | 6 | 8 | 2 | 0 |



# Floyd-warshall algorithm

```
void floydWarshall(int graph[][V]) {  
    int dist[V][V];  
    for (int i = 0; i < V; i++)  
        for (int j = 0; j < V; j++)  
            dist[i][j] = graph[i][j];  
  
    for (int k = 0; k < V; k++) {  
        for (int i = 0; i < V; i++) {  
            for (int j = 0; j < V; j++) {  
                if (dist[i][k] + dist[k][j] < dist[i][j]) {  
                    dist[i][j] = dist[i][k] + dist[k][j];  
                }  
            }  
        }  
    }  
}
```

$O(n^3)$

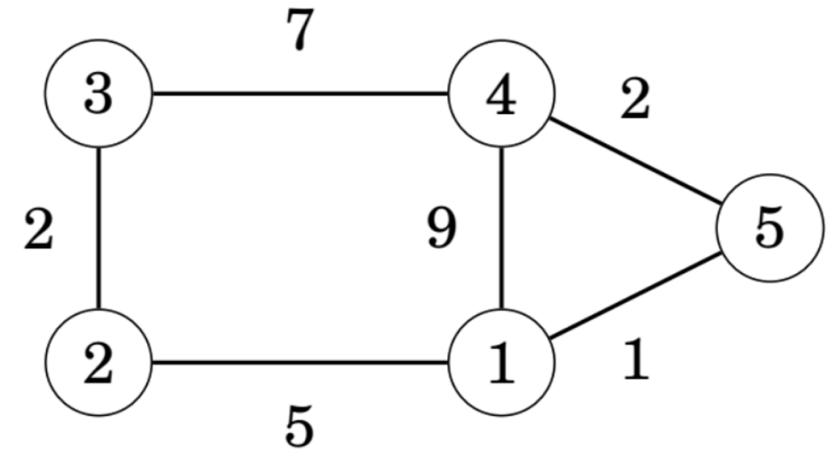
# การเก็บ path คำตอบ

สร้างเมตริกซ์ **path** เพิ่ม เพื่อเก็บ **vertex** ถัดไปที่  
จะต้องผ่านเมื่อเดินจาก **u** ไป **v**

```
for(i=0; i<V; i++)  
{ for(j=0; j<V; j++)  
  { if(graph[i][j] != INF)  
    path[i][j] = j;  
    else  
      path[i][j] = -1;  
  }  
}
```

ขณะปรับปรุง **dist[i][j]** หากพบว่าผ่าน **k** แล้วเส้นทางสั้นกว่าเดิม ก็จะปรับปรุง  
**path** ตามไปด้วย

```
if (dist[i][k] + dist[k][j] < dist[i][j]) {  
  dist[i][j] = dist[i][k] + dist[k][j];  
  path[i][j] = path[i][k];  
}
```



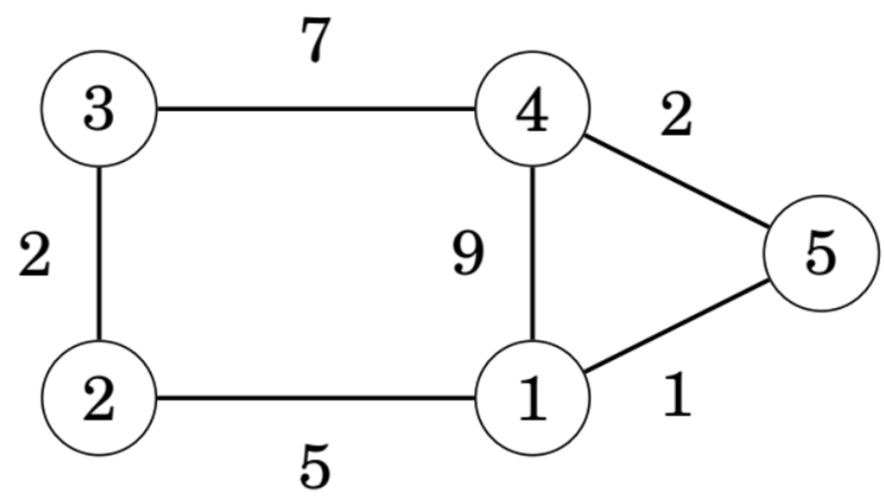
|   | 1        | 2        | 3        | 4        | 5        |
|---|----------|----------|----------|----------|----------|
| 1 | 0        | 5        | $\infty$ | 9        | 1        |
| 2 | 5        | 0        | 2        | $\infty$ | $\infty$ |
| 3 | $\infty$ | 2        | 0        | 7        | $\infty$ |
| 4 | 9        | $\infty$ | 7        | 0        | 2        |
| 5 | 1        | $\infty$ | $\infty$ | 2        | 0        |

Dist

|   | 1  | 2  | 3  | 4  | 5  |
|---|----|----|----|----|----|
| 1 | 1  | 2  | -1 | 4  | 5  |
| 2 | 1  | 2  | 3  | -1 | -1 |
| 3 | -1 | 2  | 3  | 4  | -1 |
| 4 | 1  | -1 | 3  | 4  | 5  |
| 5 | 5  | -1 | -1 | 4  | 5  |

Path

ตัวอย่าง



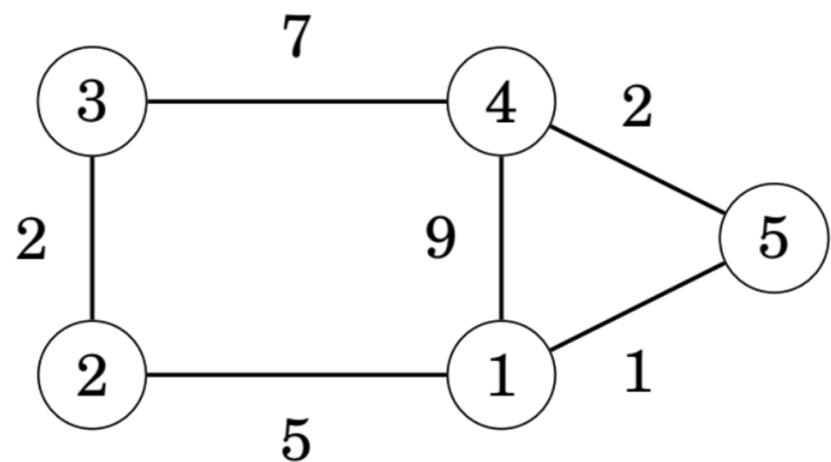
path

|   | 1  | 2  | 3  | 4  | 5  |
|---|----|----|----|----|----|
| 1 | 1  | 2  | -1 | 4  | 5  |
| 2 | 1  | 2  | 3  | -1 | -1 |
| 3 | -1 | 2  | 3  | 4  | -1 |
| 4 | 1  | -1 | 3  | 4  | 5  |
| 5 | 5  | -1 | -1 | 4  | 5  |

|   | 1        | 2        | 3        | 4        | 5        |
|---|----------|----------|----------|----------|----------|
| 1 | 0        | 5        | $\infty$ | 9        | 1        |
| 2 | 5        | 0        | 2        | $\infty$ | $\infty$ |
| 3 | $\infty$ | 2        | 0        | 7        | $\infty$ |
| 4 | 9        | $\infty$ | 7        | 0        | 2        |
| 5 | 1        | $\infty$ | $\infty$ | 2        | 0        |

graph

# ตัวอย่าง $k = 1$



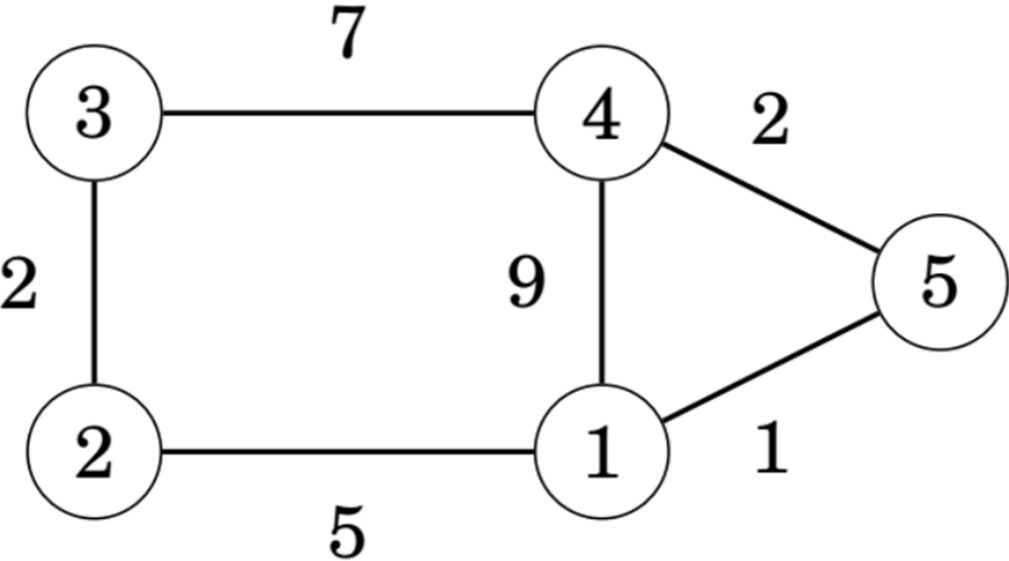
path

|   | 1  | 2 | 3  | 4 | 5  |
|---|----|---|----|---|----|
| 1 | 1  | 2 | -1 | 4 | 5  |
| 2 | 1  | 2 | 3  | 1 | 1  |
| 3 | -1 | 2 | 3  | 4 | -1 |
| 4 | 1  | 1 | 3  | 4 | 5  |
| 5 | 5  | 1 | -1 | 4 | 5  |

|   | 1        | 2        | 3        | 4        | 5        |
|---|----------|----------|----------|----------|----------|
| 1 | 0        | 5        | $\infty$ | 9        | 1        |
| 2 | 5        | 0        | 2        | $\infty$ | $\infty$ |
| 3 | $\infty$ | 2        | 0        | 7        | $\infty$ |
| 4 | 9        | $\infty$ | 7        | 0        | 2        |
| 5 | 1        | $\infty$ | $\infty$ | 2        | 0        |

|   | 1        | 2  | 3        | 4  | 5        |
|---|----------|----|----------|----|----------|
| 1 | 0        | 5  | $\infty$ | 9  | 1        |
| 2 | 5        | 0  | 2        | 14 | 6        |
| 3 | $\infty$ | 2  | 0        | 7  | $\infty$ |
| 4 | 9        | 14 | 7        | 0  | 2        |
| 5 | 1        | 6  | $\infty$ | 2  | 0        |

# ตัวอย่าง $k = 2$



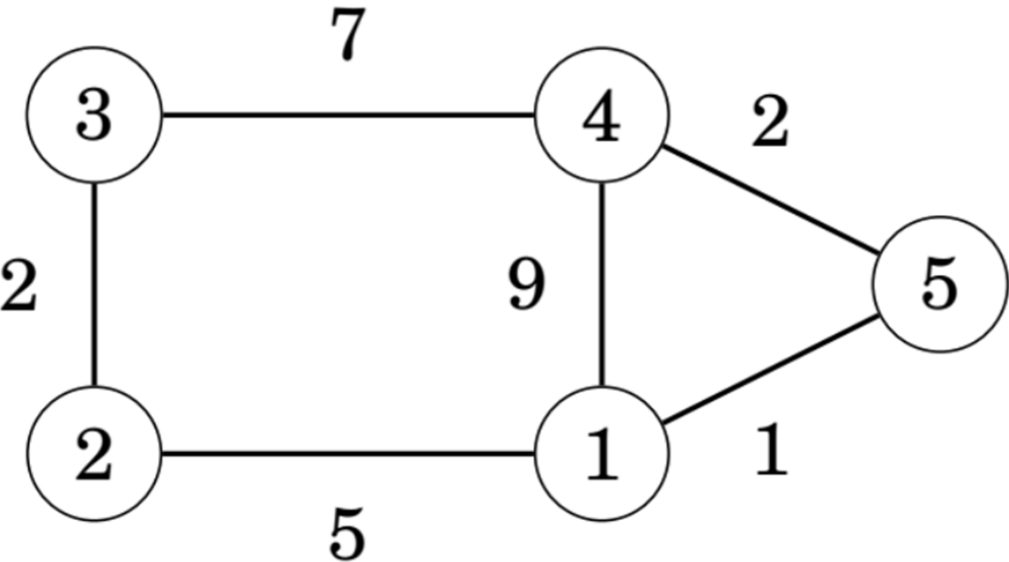
path

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 1 | 2 | 2 | 4 | 5 |
| 2 | 1 | 2 | 3 | 1 | 1 |
| 3 | 2 | 2 | 3 | 4 | 2 |
| 4 | 1 | 1 | 3 | 4 | 5 |
| 5 | 5 | 1 | 2 | 4 | 5 |

|   | 1        | 2  | 3        | 4  | 5        |
|---|----------|----|----------|----|----------|
| 1 | 0        | 5  | $\infty$ | 9  | 1        |
| 2 | 5        | 0  | 2        | 14 | 6        |
| 3 | $\infty$ | 2  | 0        | 7  | $\infty$ |
| 4 | 9        | 14 | 7        | 0  | 2        |
| 5 | 1        | 6  | $\infty$ | 2  | 0        |

|   | 1 | 2  | 3 | 4  | 5 |
|---|---|----|---|----|---|
| 1 | 0 | 5  | 7 | 9  | 1 |
| 2 | 5 | 0  | 2 | 14 | 6 |
| 3 | 7 | 2  | 0 | 7  | 8 |
| 4 | 9 | 14 | 7 | 0  | 2 |
| 5 | 1 | 6  | 8 | 2  | 0 |

# ตัวอย่าง $k = 3$



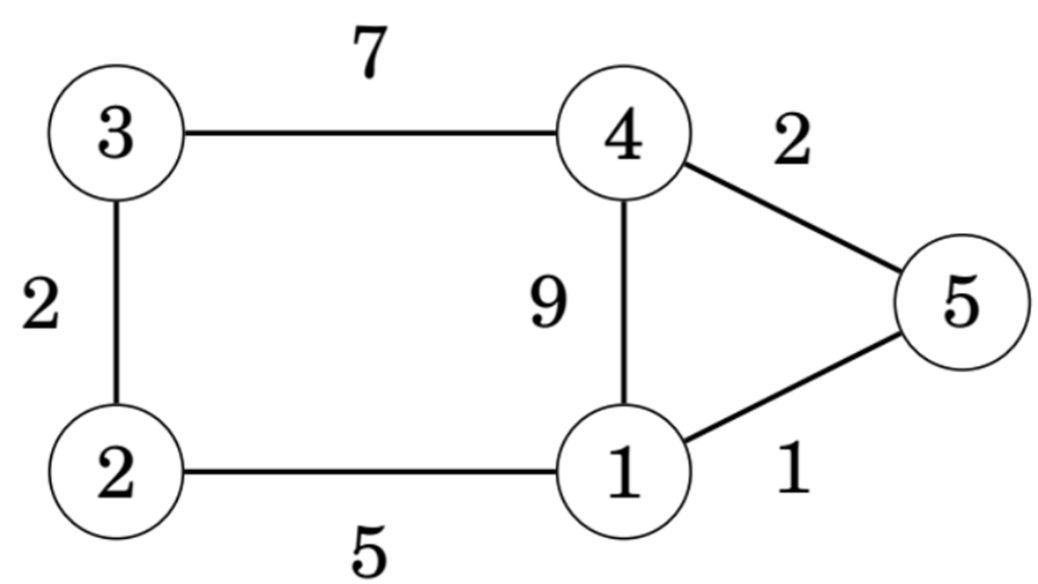
path

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 1 | 2 | 2 | 4 | 5 |
| 2 | 1 | 2 | 3 | 3 | 1 |
| 3 | 2 | 2 | 3 | 4 | 2 |
| 4 | 1 | 3 | 3 | 4 | 5 |
| 5 | 5 | 1 | 2 | 4 | 5 |

|   | 1 | 2  | 3 | 4  | 5 |
|---|---|----|---|----|---|
| 1 | 0 | 5  | 7 | 9  | 1 |
| 2 | 5 | 0  | 2 | 14 | 6 |
| 3 | 7 | 2  | 0 | 7  | 8 |
| 4 | 9 | 14 | 7 | 0  | 2 |
| 5 | 1 | 6  | 8 | 2  | 0 |

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 5 | 7 | 9 | 1 |
| 2 | 5 | 0 | 2 | 9 | 6 |
| 3 | 7 | 2 | 0 | 7 | 8 |
| 4 | 9 | 9 | 7 | 0 | 2 |
| 5 | 1 | 6 | 8 | 2 | 0 |

# ตัวอย่าง $k = 4$



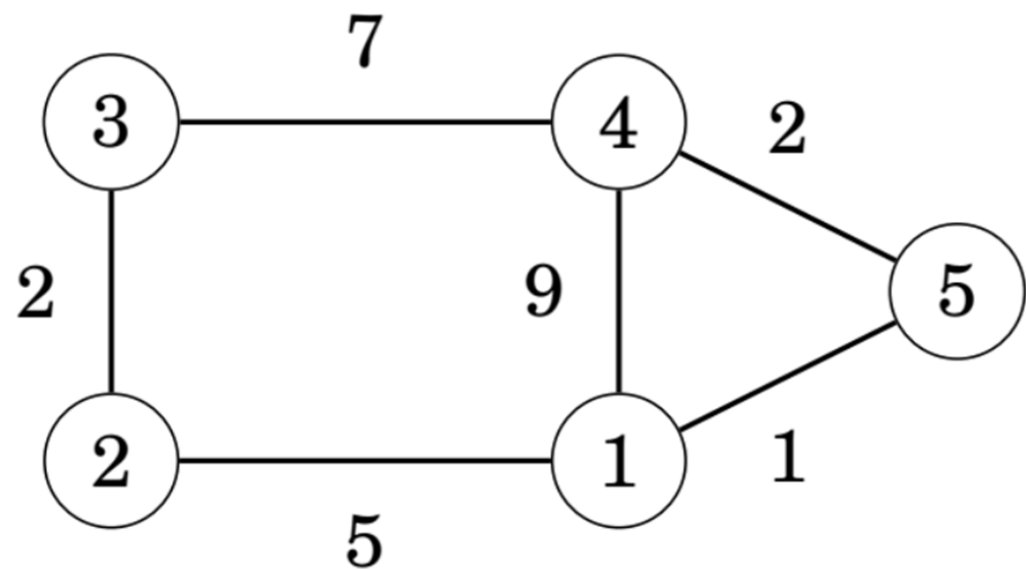
path

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 1 | 2 | 2 | 4 | 5 |
| 2 | 1 | 2 | 3 | 3 | 1 |
| 3 | 2 | 2 | 3 | 4 | 2 |
| 4 | 1 | 3 | 3 | 4 | 5 |
| 5 | 5 | 1 | 2 | 4 | 5 |

|   | 1        | 2        | 3 | 4        | 5 |
|---|----------|----------|---|----------|---|
| 1 | 0        | 5        | 7 | 9        | 1 |
| 2 | 5        | 0        | 2 | <b>9</b> | 6 |
| 3 | 7        | 2        | 0 | 7        | 8 |
| 4 | <b>9</b> | <b>9</b> | 7 | 0        | 2 |
| 5 | 1        | 6        | 8 | 2        | 0 |

|   | 1 | 2        | 3 | 4        | 5 |
|---|---|----------|---|----------|---|
| 1 | 0 | 5        | 7 | 9        | 1 |
| 2 | 5 | 0        | 2 | <b>9</b> | 6 |
| 3 | 7 | 2        | 0 | 7        | 8 |
| 4 | 9 | <b>9</b> | 7 | 0        | 2 |
| 5 | 1 | 6        | 8 | 2        | 0 |

# ตัวอย่าง $k = 5$



path

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 1 | 2 | 2 | 5 | 5 |
| 2 | 1 | 2 | 3 | 5 | 1 |
| 3 | 2 | 2 | 3 | 4 | 2 |
| 4 | 5 | 5 | 3 | 4 | 5 |
| 5 | 5 | 1 | 2 | 4 | 5 |

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 5 | 7 | 9 | 1 |
| 2 | 5 | 0 | 2 | 9 | 6 |
| 3 | 7 | 2 | 0 | 7 | 8 |
| 4 | 9 | 9 | 7 | 0 | 2 |
| 5 | 1 | 6 | 8 | 2 | 0 |

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 5 | 7 | 3 | 1 |
| 2 | 5 | 0 | 2 | 8 | 6 |
| 3 | 7 | 2 | 0 | 7 | 8 |
| 4 | 3 | 8 | 7 | 0 | 2 |
| 5 | 1 | 6 | 8 | 2 | 0 |



# Reconstruction of shortest path

```
void printPath(int u, int v, int path[V][V])
{ if(path[u][v] == -1)
  { printf("No path\n");
    return;
  }
  printf("Path: %d", u);
  while(u != v)
  { u = path[u][v];
    printf(" -> %d", u);
  }
  printf("\n");
}
```

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 1 | 2 | 2 | 5 | 5 |
| 2 | 1 | 2 | 3 | 5 | 1 |
| 3 | 2 | 2 | 3 | 4 | 2 |
| 4 | 5 | 5 | 3 | 4 | 5 |
| 5 | 5 | 1 | 2 | 4 | 5 |

```
printPath(1, 4, path)
1 -> 5 -> 4
```